SHL Assessment Recommendation System

Approach Document

Author: Mohammad Inayat Hussain

Page 1: Problem, Architecture and Data Pipeline

Problem Statement

Given a natural-language job description or a hiring query, recommend the most relevant SHL assessments from the product catalog. The system needs to expose a REST API with both GET and POST endpoints, return results as JSON, and generate a CSV of predictions for batch evaluation.

Architecture

The pipeline is simple and sequential:

User Query -> Sanitize -> Extract Constraints -> Expand with Gemini LLM

-> Embed wit

I split scoring into three parts because no single method worked well enough on its own:

- Semantic similarity at 50 percent weight. Cosine similarity between the query embedding and precomputed assessment embeddings using all-MiniLM-L6-v2. This handles the broad intent of the query.

- Keyword overlap at 25 percent weight. Token intersection between query words and assessment text after removing stop words. This catches exact technology names like Java or SQL that embeddings tend to blur together.

- Assessment name match at 25 percent weight. Direct token overlap with assessment titles. When someone searches for Java 8 the assessment literally named Java 8 New gets a strong boost instead of being buried under generic coding tests.

The system also extracts constraints from the query before scoring:

- Time limits

Data Pipeline

Scraping. I built a scraper with requests and BeautifulSoup that paginates through the SHL product catalog filtering for Individual Test Solutions. It visits each detail page to pull descriptions, test types, durations, remote and adaptive support, and job levels. I added retry logic and polite delays because SHL occasionally returns 5xx errors.

Cleaning. The raw scraper output needed work. Test types were space-separated on the site like C P A B so I converted them to comma-separated. Missing durations got set to -1 as a sentinel value. Unicode dashes were breaking name matching so I normalized them. The cleaning script also handles deduplication, URL validation, and builds a combined text column that the embedding model uses.

Final catalog is 389 Individual Test Solutions with structured metadata.

Tech Stack

API: FastAPI plus Uvicorn

Embeddings:

Page 2: Evaluation, Decisions and Security

Evaluation Journey

The primary metric is Mean Recall at 10 against the labeled training set which has 10 queries each with 1 to 3 ground truth assessments.

Iteration 1: Baseline semantic search only. Mean Recall at 10 was 0.567.

Iteration 2: Ac

The biggest win in the last iteration was injecting domain-specific synonyms before embedding. For example expanding behavioral to OPQ personality questionnaire bridges the vocabulary gap between how people ask questions and how SHL names its products. The adaptive boost and name matching each added smaller but meaningful improvements.

Design Decisions

Why hybrid instead of pure semantic search. Pure embedding search missed exact technology names. A query for Java 8 would return general coding tests because the embedding space does not differentiate well between specific tech stacks. Adding keyword overlap and name matching fixed this.

Why Gemini is optional. The system works without an API key by falling back to raw queries. This means the evaluation pipeline never fails due to LLM quota limits or network issues.

Why no external vector database. With only 389 assessments a NumPy matrix fits easily in memory and cosine similarity runs in under 50 milliseconds. Adding Pinecone or ChromaDB would be unnecessary complexity for this scale.

Security Measures

These are implemented and active in the deployed system:

Input sanitization. All queries pass through bleach to strip HTML and script tags, null bytes, and excessive whitespace. Queries are capped at 10000 characters.

SSRF protection. URL inputs are validated against private and internal IP ranges including 127.0.0.1, 169.254.169.254, and the 10.x, 172.16.x, 192.168.x blocks to prevent cloud metadata attacks.

Rate limiting. slowapi enforces 100 requests per minute per IP address.

Error masking. A global exception handler returns generic 500 responses with a trace ID. No stack traces or file paths leak to the client.

Security headers. Every response includes X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, and HSTS headers.

For production the next steps would be adding JWT or OAuth2 authentication, restricting CORS to known frontend domains, and setting up structured audit logging.

Known Limitations

LinkedIn blocks automated requests so pasted LinkedIn URLs fall back to extracting keywords from the URL slug which is less accurate than having the full job description text.

Very short or vague queries like just the word test do not give the engine enough signal. It still returns results but relevance drops noticeably.

Future Work

Fine-tune embeddings on SHL-specific assessment and query pairs to improve domain understanding.

Add a cross-e